Certified Federated Unlearning for IoT: Ledger-Anchored Revocation with Anytime-Valid Forgetting Certificates

Quang-Vinh Dang

***Abstract*—Federated learning lets IoT devices train a shared model without exporting data, but a device's influence stays in the model after it leaves, and regulation, decommissioning and poisoned contributions all demand removal. Retraining for every request is too costly for constrained fleets, and existing federated unlearning checks are empirical, intrusive or limited to logging the request. We add an unlearning and governance layer to the FORTRESS-FL robust aggregation system. A hash-chained ledger, per-round checkpoints of model and reputation state, and a consent registry make revocation traceable. Clients plant paired ghost canaries, and an anytime-valid e-process verifier (VOUCH) certifies a revocation wave without retrained references or backdoor markers. On top of this we contribute incremental certified recovery, which audits after each block of recovery rounds under a union-bound error budget and never costs more client rounds than retraining; entanglement-targeted recovery driven by the ledger's logged updates; a power check, canary calibration and real-record twin audit; and a deadline-aware wave scheduler. On a synthetic fleet, smartphone sensor data (UCI HAR) and botnet traffic from nine real IoT devices (N-BaIoT), with a linear model and an MLP, exactly retrained models were wrongly flagged in 4 of 280 runs. Incremental recovery used 60–97% less client energy than retraining and 0–59% less than a certified ladder, targeting saved up to 83% more, and marker-based and membership-inference baselines did not separate leaky from exact models. Batching cut energy per request 12-fold at 1 request per day.**

***Index Terms*—Federated unlearning, model governance, provenance, anytime-valid inference, Byzantine robustness, Internet of Things.**

I. Introduction

FEDERATED learning lets IoT devices train a shared model without exporting raw data [1], but a device's data keeps influencing the global model long after the device has left. Data protection law gives owners a right to erasure [2], devices are decommissioned or change owner, and poisoned contributions are discovered after the fact. Retraining from scratch for every request costs the whole fleet its original training energy and radio traffic, which is not realistic at IoT scale.

Federated unlearning removes a client's influence without full retraining [3]–[6]. Two problems remain for deployment. First, the operator has to *show* that the influence is gone. Existing checks mark the leaving client's data with backdoor-style markers [7], rely on fingerprints [8], or record requests on a blockchain [9], [10]; the first two are empirical and the third records that unlearning was requested rather than whether it worked. Second, verification is almost never tied to cost: an operator who must pay for a certificate has little reason to choose the cheap method over the expensive one unless the certificate tells them whether the cheap method was enough.

Q.-V. Dang is with RMIT University Vietnam. E-mail: vinh.dang3@rmit.edu.vn.

We address both problems in one system built on the FORTRESS-FL robust aggregation pipeline [11] and on the paired-canary verifier VOUCH [12]. Our contributions are:

1) **Ledger-anchored revocation.** A hash-chained provenance ledger with per-round Merkle roots over client updates, a consent registry, and per-round checkpoints of the model *and* of the reputation state, since reputations also remember past contributors. A rollback record names the clean checkpoint, so an auditor can check lineage and bind a certificate to the ledger head and the released model hash.
2) **Federated paired-canary certificates.** Clients plant paired ghost canaries before training and commit to them in the ledger; after a revocation wave an anytime-valid e-process verifier [12] issues or withholds an (ϵ, α) certificate without retrained references or backdoor markers. We adapt the protocol to client-level revocation, partial participation and batched waves.
3) **Incremental certified recovery (ICR).** Recovery runs on one model in blocks and the audit is repeated after each block at level $\alpha/(k(k+1))$, so a failed audit wastes no work, the error budget holds over the whole run (Theorem 2), and the client cost never exceeds retraining. **Entanglement-targeted recovery** restricts recovery to the retained clients whose logged updates align most with the revoked ones.
4) **Audit design.** A two-tier audit with canaries (worst case) and paired real-record twins (average case); a power check and a canary calibration that report when the audit cannot see leakage; and a deadline-aware wave scheduler that trades latency for audit power and energy.
5) **Evaluation on resource-constrained fleets.** We quantify client energy and traffic under an explicit cost model, on a synthetic fleet, smartphone sensor data (UCI HAR [13]) and real IoT botnet traffic (N-BaIoT [14]), with a linear model and an MLP, against retraining, a certified ladder, a marker-based verifier and a membership-inference test.

What is new relative to FORTRESS-FL and VOUCH

This paper builds on two earlier works by the same author, and we state precisely what it adds. *FORTRESS-FL* [11] is a training-time system: Pedersen-committed updates,

TABLE I
CONTRIBUTION OF EACH WORK.

	FORTRESS-FL [11]	VOUCH [12]	This paper
Setting	federated training	centralised LMs	federated, IoT
Byzantine robust	yes	n/a	inherited, tested during unlearning
Client removal	no	forget-set only	client waves, operators
Per-round state	no	no	model, reputation, updates
Provenance	commitments	manifest hash	ledger, Merkle, consent
Verification	none	e-process certificate	certificate per wave, ledger-bound
Cost-aware choice	no	no	ICR, ladder, scheduler
Energy/traffic	no	GPU passes	client model

spectral Byzantine detection, reputation-weighted aggregation and Gaussian noise. It has no notion of removing a client after training, keeps no per-round state, and offers no audit. *VOUCH* [12] is a verifier for centralised (language-model) unlearning: it assumes a single trained model and a forget set, and certifies it with paired canaries and e-processes. It has no federated protocol, no client revocation, no unlearning operators, and no cost model. Table I lists what each contributes and what is added here. The new elements are: (i) checkpointing of model *and* reputation state, the ledger, the consent registry and the unlearning operators on top of FORTRESS-FL; (ii) a federated canary protocol in which clients hold and commit their own pairs and certificates are issued per revocation wave under partial participation; (iii) incremental certified recovery, entanglement-targeted recovery and the verify-then-escalate ladder, with their error accounting (Theorems 1 and 2); (iv) the real-record twin audit, the power check and canary calibration, and the wave scheduler; (v) the energy and traffic model; and (vi) experiments on three data sets, two models, and baselines. The e-process verifier and its coverage theorem are used unchanged from VOUCH and are not claimed as new.

II. RELATED WORK

Federated unlearning. FedEraser rebuilds the model from stored historical updates with calibration [3]; other methods use knowledge distillation [5], gradient-based removal [4], noise-based certified removal [15] or on-device teachers [16]. FedRecover estimates client updates to recover from poisoning [17]. Exact approaches isolate data in shards [18]. Surveys [6] note that cost, verification and non-IID behaviour are open. Our operators (subtraction of stored updates, rollback to the last clean checkpoint, replay, sharding) are standard; the contribution is the certificate that decides between them.

Verifying unlearning. Backdoor-marker verification in federated settings [7] detects removal through the behaviour of the marker, which requires modifying the data and gives no error guarantee. PrivEraserVerify reports fingerprint-based verification with indistinguishability claims [8]. Membership inference is the usual statistical probe [19], [20], but a threshold on an attack score is not a calibrated test, and unlearning definitions need to be auditable [21]. Certified removal gives guarantees for particular algorithms and convex models [22]; ours is a property of the released model, whichever algorithm produced it.

Provenance and ledgers in FL. Blockchain-based designs record updates and unlearning events for traceability [9], [10]. We use a lightweight hash chain with Merkle inclusion proofs [23] and, unlike a log of requests, couple it to a statistical test of the result.

Anytime-valid testing. E-processes and betting martingales give tests valid under optional stopping [24], [25]. VOUCH applies them to centralised unlearning with paired canaries [12]. We reuse its verifier and theorems as a component and cite them rather than restate them; the federated protocol, ladder, ledger and evaluation are new here.

III. PROBLEM AND THREAT MODEL

N clients, each with a local data set $\mathcal{D}_c$, train a model $\theta \in \mathbb{R}^d$ with T rounds of robust aggregation. In round t a subset of clients participates, each independently with probability p. A *revocation wave* $\mathcal{W}$ is a set of clients whose consent is withdrawn (or whose updates are found poisoned). The operator must release a model θ_u that no longer carries their influence, and produce evidence that a third party can check.

The operator is honest-but-verifiable: it executes the protocol but may take shortcuts to save cost, which is exactly the temptation the audit has to catch. A minority of clients may be Byzantine during training. Clients are not trusted to self-report forgetting. What is certified is the *residual advantage* that a declared class of scores can extract from the canary cohort of the wave on the released model; it is not erasure in parameter space, which no behavioural test can establish.

IV. SYSTEM

A. Training and provenance

Training follows FORTRESS-FL: each client commits to its clipped update with a Pedersen commitment [26], the server verifies openings, clusters updates spectrally to flag a Byzantine group, aggregates the rest with reputation weights, adds Gaussian noise and updates reputations [27], [28]. We gate the cluster split: a group is flagged only if its cross-similarity to the rest is lower than the within-group similarity by more than a margin, so IID honest rounds are not split arbitrarily.

After each round the ledger appends a record containing the round index, the participant list, the Merkle root over leaves $H(\text{client} \,\|\, H(\text{update}))$, the aggregation weights, the flagged clients and the hash of the new model. Each record includes the hash of its predecessor. The server also stores the model and the reputation vector of every round (the checkpoint), and, optionally, the clipped updates themselves. A *consent registry* tracks revocations and marks them fulfilled only after a rollback record exists.

B. Unlearning operators and cost model

For a wave $\mathcal{W}$ let k_0 be the first round in which any member of $\mathcal{W}$ participated according to the ledger, and θ_{k_0-1} its checkpoint. The operators are: *retrain* (restart from θ_0 on the retained clients); *rollback* (restore θ_{k_0-1} and the reputations of that round, then re-run rounds $k_0, \dots, T$ without $\mathcal{W}$); *subtract* ($\theta_T + \eta \sum_t \sum_{c\in\mathcal{W}} w_{c,t}\tilde{g}_{c,t}$ using the stored clipped updates, and zero reputation); *subtract+R* (subtract, then R fresh rounds); *replay* (re-apply stored updates of the retained clients from the checkpoint, no client computation); and *sharded* (clients split into cohorts, one cohort retrained). After a rollback the ledger forks off the contaminated checkpoints and stored updates are purged so that the revoked clients' influence does not survive in the server's own records.

Proposition 1. *If clients participate independently with probability p in each round, the first round k_0 in which any of m revoked clients participates satisfies $\Pr[k_0 > r] = (1-p)^{rm}$, so $\mathbb{E}[k_0] = 1/(1-(1-p)^m)$ (about 1 for $m \geq 5$ at $p = 0.6$) and rollback saves at most about one round out of T.*

This explains why rollback matched retraining in our runs and why savings must come from approximate operators that are then verified. Batching helps differently: rollback cost does not grow with $|\mathcal{W}|$, so the cost per revoked client falls as $1/|\mathcal{W}|$.

Cost model. A client round costs $4n_c d$ FLOPs, uploads $4d + 64$ bytes (float32 update and a 512-bit commitment) and downloads $4d$ bytes. Energy is $e_{\text{flop}} \cdot \text{FLOPs} + e_{\text{bit}} \cdot 8 \cdot \text{bytes}$ with $e_{\text{flop}} = 1$ nJ and $e_{\text{bit}} = 100$ nJ. These constants are assumptions typical of low-power radios and processors, not measurements; the ratio between methods follows the round count and is insensitive to them. Canary reveal adds $2m(d+1) \cdot 4$ bytes per revoked client once.

C. Paired canaries and the certificate

Before training, each client draws m twin pairs of exchangeable records (Gaussian features, fair random labels), flips a coin per pair to pick the in-twin, adds only the in-twin to its data, keeps the ghost, and publishes a digest of the manifest in the ledger. After a wave, the verifier receives the pairs of $\mathcal{W}$ and scores the released model on both twins with a declared class of scores (negative loss and signed margin). If θ_u carries no influence of the in-twins, each sign of the score difference is a fair coin; betting e-processes on these signs yield (i) a confidence sequence for the residual advantage Δ, (ii) an (ϵ, α) certificate once evidence that $|\Delta| < \epsilon$ is conclusive, and (iii) a revocation alarm when the advantage is positive [12]. In a deployment the reveal order of the pairs is drawn from a public beacon; our experiments use a fixed seed so that runs are reproducible.

D. Verify-then-escalate

Let rungs $1, \dots, K$ be operators ordered by cost. The policy runs rung k on a scratch copy, tests its output at level α/K, and releases the first rung that is certified and keeps validation accuracy within δ of the pre-unlearning model. If the last rung does not certify, it is released flagged UNCERTIFIED. Before the first rung the policy audits the model before unlearning; if that audit does not flag it, the canaries were not memorised, a certificate would hold vacuously, and the final status is reported as UNPOWERED instead of ISSUED.

Theorem 1. *Let L be the set of rungs whose output has realised cohort advantage at least ϵ in some score of the declared class. The probability that the policy ever releases a rung in L as certified is at most α, for any (data-dependent) stopping rule inside each rung's test and any dependence between rungs.*

Proof. By the coverage theorem of the verifier [12], each rung $k \in L$ is certified with probability at most α/K. A union bound over at most K rungs gives α. The utility guard only rejects further rungs and cannot increase the probability of releasing a rung in L. □

E. Incremental certified recovery and targeting

The ladder restarts from the checkpoint for every rung, so a failed attempt is paid for in full. ICR removes this: it audits the pre-unlearning model (power check), applies *subtract*, and then repeats: audit the current model at level $\alpha_k = \alpha/(k(k+1))$ and release it if certified and within the accuracy guard; otherwise run a block of B recovery rounds on the *same* model and continue, up to $R_{\max}$ rounds.

Theorem 2. *Let the audits after blocks $k = 1, 2, \dots$ be run at levels $\alpha_k = \alpha/(k(k+1))$. The probability that ICR ever releases as certified a model whose realised cohort advantage is at least ϵ is at most α. With $R_{\max} \leq T$ it uses at most T client rounds, the same as retraining.*

Proof. Each audit is a valid level-α_k test of its own model [12]; since $\sum_k \alpha_k = \alpha$, the union bound gives the first claim for any dependence between the successive models. The round bound is the stopping rule. □

Compared with the ladder (levels α/K for K fixed rungs), ICR needs no K in advance and spends most of the budget on early audits, where most waves stop.

Entanglement-targeted recovery. From the stored updates, the ledger gives the score $e_c = \frac{1}{|\mathcal{T}_c|} \sum_{t\in\mathcal{T}_c} \cos\big(g_{c,t}, \bar{g}_{\mathcal{W},t}\big)$ of each retained client c, where $\bar{g}_{\mathcal{W},t}$ is the mean logged update of the revoked clients in round t and $\mathcal{T}_c$ the rounds in which both took part. Recovery then uses only the top q fraction of retained clients. This cuts energy roughly in proportion to q; because the audit still decides, a targeted model is released only if certified.

F. Audit design: real twins, power and scheduling

Real-record twins. A client pairs a fraction of its own records at random, a coin decides which twin is trained and the other is withheld. The random matching makes the twins exchangeable, so the same test applies and the audit measures advantage on real records instead of planted noise; the price is withheld data. *Power check and calibration.* Before unlearning, the audit is run on the model that still contains the wave; if it is not flagged, the canaries were not memorised and the final status is UNPOWERED. Calibration picks the smallest canary count

m (then feature scale) on a pilot run for which the audit flags the pilot model. *Wave scheduler.* Requests wait until the queue holds $P^\star$ canary pairs, or until the oldest has waited $D - \delta$ days (D the legal response period, δ a margin), whichever comes first.

V. Experiments

A. Setup

Federation. $N = 60$ clients hold linear classifiers on $d = 1024$-dimensional features, the regime of a linear head on frozen on-device embeddings. Data are ten Gaussian latent classes with binary labels (class parity) and 8% label noise, split by a Dirichlet(α=1) partition; each client has 12 real samples and plants $m = 6$ canary pairs. Each round every client participates with $p = 0.6$; $T = 60$ rounds, learning rate 1, update clipping at norm 1, no DP noise unless stated. A wave revokes $W = 20$ clients (120 pairs); $\epsilon = 0.2$, $\alpha = 0.05$. Test accuracy of the trained model is 0.98. Main results use 10 seeds; sweeps, wave-size and poisoning experiments use 5. All numbers are regenerated by `experiments/unlearning_experiments.py`. The audit's reveal order is seeded for reproducibility (a deployment would use a beacon). Real-data experiments with an MLP are in Section V-I; the sweeps use the synthetic fleet only. Hyper-parameters, the cost constants, further figures (S1–S6), all result tables (S1–S8) and reproduction commands are in the Supplementary Material.

Cost. Energy and traffic come from the model in Section IV-B. With the assumed constants radio traffic dominates energy, so energy tracks bytes.

B. Calibration of the audit

We first check the verifier on the two cases where the truth is known (40 seeds, N=40, T=30, W=20). The model before unlearning was flagged REVOKED in 40 of 40 runs (upper bound on the advantage 0.78), and the exactly retrained model was ISSUED in 40 of 40 runs with no false revocation. A leaky model was therefore never certified and an exactly unlearned one was never accused.

C. Unlearning operators under the certificate

Table II and Fig. 1 compare the operators on the same wave. The un-unlearned model is flagged in 10 of 10 runs. Rollback restored round 0 in every run, because with 20 revoked clients and $p = 0.6$ some member participates in round 1 (Proposition 1), so rollback and retraining are the same here. Plain subtraction and replay cost no client energy but cost accuracy (0.62), and the certificate cannot see that: it was issued in 4 of 10 runs, which is why the policy needs the utility guard. Subtraction followed by R recovery rounds restores accuracy (0.98); more recovery rounds raise the share of certified runs (4, 4 and 6 of 10 for $R = 5, 10, 20$, and 8 of 10 for retraining) at 0.8, 1.6, 3.2 and 9.6 J. No operator was certified in all runs: with 120 pairs the test is near its resolution at $\epsilon = 0.2$, and even retraining ended UNDETERMINED twice. UNDETERMINED means the evidence was not conclusive, not that the model leaks.

TABLE II
OPERATORS ON ONE WAVE OF 20 CLIENTS (10 SEEDS). I/U/R: RUNS WITH CERTIFICATE ISSUED, UNDETERMINED, REVOKED. ENERGY IS CLIENT-SIDE.

Operator	I/U/R	Accuracy	Rounds	Energy (J)
none	0/0/10	0.98	0	0
subtract	4/1/5	0.62	0	0
replay	4/1/5	0.62	0	0
subtract+5	4/1/5	0.98	5	0.81
subtract+10	4/3/3	0.98	10	1.64
subtract+20	6/3/1	0.98	20	3.24
rollback	8/2/0	0.98	60	9.58
retrain	8/2/0	0.98	60	9.58

D. Verify-then-escalate

The ladder was subtract, subtract+5, subtract+20, rollback, each tested at $\alpha/4$, with a validation set of 300 samples and a 3-point accuracy guard. It certified 10 of 10 waves at accuracy 0.98, against 8 of 10 for always retraining at the full level. It released subtract+5 four times, subtract+20 twice and rollback four times. Mean client energy was 6.6 J against 9.6 J for retraining (31% less, 41 instead of 60 rounds on average), but the cost is bimodal: from 0.8 J when the first rung succeeded to 13.8 J, 44% above retraining, when all four rungs were tried because failed attempts are paid for. Fixing subtract+5 as the only operator was cheapest (0.8 J) and was certified in 4 of 10 runs only (Supplement, Fig. S1).

E. Wave size and batching

Retraining cost per revoked client falls from 3.4 J for waves of 4 to 0.24 J for waves of 30, because rollback cost does not grow with the wave. Serving 20 requests one at a time costs 14.0 J each (measured on three requests and extrapolated), 29 times more than a single batched wave. The price is delay and the need for enough pairs: the retrained model was certified in 3 of 5 runs at W=4 (24 pairs) and in 5 of 5 at $W \in \{8, 20, 30\}$, with 4 of 5 at W=12. Revealing the canaries sends $2m(d{+}1){\cdot}4$ bytes per client, 0.98 MB for W=20. (Supplement, Fig. S2.) In practice requests have to be held until a wave is large enough, which is compatible with the response periods of data protection law but should be agreed with the regulator.

F. Poisoned and Byzantine clients

Three or eight clients in the wave stamped a trigger on half of their data (relabelled to the target class); in half of the runs 12 further clients were Byzantine sign-flippers. Before unlearning the backdoor attack success rate was 0.29 with three poisoned clients and 0.71 with eight (0.31 and 0.71 with Byzantine clients present). After subtraction with 10 recovery rounds it fell to 0.007 and 0.002 (0.016 and 0.010 with Byzantine clients), against 0.027 for retraining and 0.029 for a model never trained on them, at 2.1–2.3 J versus 12.5–13.8 J, with accuracy unchanged. The rate of the clean model is the false-trigger floor. Recovery was as effective with the Byzantine clients present (Supplement, Fig. S3).

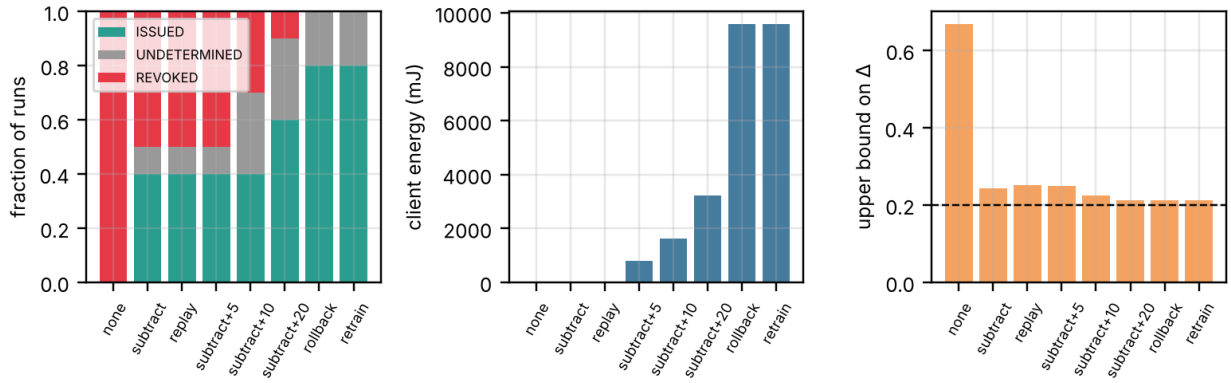


Fig. 1. Certificate outcomes, client energy and upper confidence bound on the residual advantage per operator (10 seeds). The dashed line is $\epsilon = 0.2$.

G. Heterogeneity, participation, noise and capacity

Over four factors (5 seeds), the approximate operators were certified more often with stronger heterogeneity (subtract+10: 5, 2 and 3 of 5 runs at $\alpha = 0.1, 1, 10$), lower participation (4, 2 and 3 of 5 at $p = 0.3, 0.6, 1$) and more DP noise (2 and 5 of 5 at $\sigma = 0$ and 0.02). We did not isolate the cause; fewer exposures of a canary and noise on the update both plausibly reduce what the model retains. Accuracy fell with noise (0.98 to 0.94 at $\sigma = 0.02$) for every operator (Supplement, Fig. S4). Capacity matters for audit power: at $d = 64$ the un-unlearned model was certified in 3 of 5 runs, because the model cannot memorise the canaries, so the advantage really is below ϵ there; at $d \geq 256$ it was flagged in all runs.

H. Provenance overhead and tamper detection

For 60 rounds the ledger holds 61 records and 101 kB (1.7 kB per round), about 1.1% of update upload traffic. Verifying the whole chain took 1.6 ms and a Merkle inclusion proof (5–6 hashes) about 5 μs. In 3 seeds $\times$ 200 random edits of a record field (root, model hash, round, weights, flagged list), the chain check detected all 600; an HMAC-signed certificate failed on any change of its status; a forged rollback record that kept training the revoked wave was rejected by the lineage audit; and the lineage audit passed for every operator in the main experiment. The server's checkpoints took 417 kB, but stored updates 15.2 MB, which is the main storage cost of the cheap operators.

I. Real IoT data and a nonlinear model

Data and setup. UCI HAR [13] gives 561 smartphone inertial features for 30 subjects, one client per subject (natural non-IID split; task: dynamic versus static activity). N-BaIoT [14] gives 115 traffic statistics from nine commercial IoT devices infected with Mirai and BASHLITE; we split each device into five sites with disjoint random subsets of its traffic, giving 45 clients that differ by device type and attack family (task: benign versus attack). Every client keeps 40 records, half benign for N-BaIoT, and plants $m = 6$ canary pairs; test records are held out per client. We train a logistic model and a one-hidden-layer MLP (32 tanh units; 18 049 parameters on HAR, 3 777 on N-BaIoT), $T = 60$, $p = 0.6$, and revoke a third of the clients (10 of 30 and 15 of 45; 60 and 90 pairs), 5 seeds, $\epsilon = 0.2$. Accuracy is near ceiling (0.997–0.999), so the certificate carries the evidence.

TABLE III
CERTIFICATE OUTCOMES I/U/R (ISSUED, UNDETERMINED, REVOKED) ON REAL DATA, WAVE OF ONE THIRD OF THE CLIENTS, 5 SEEDS.

	HAR		N-BaIoT	
Operator	Linear	MLP	Linear	MLP
none	0/0/5	0/0/5	3/1/1	0/0/5
subtract	2/2/1	3/2/0	4/1/0	5/0/0
replay	2/1/2	3/2/0	4/1/0	4/1/0
subtract+5	2/2/1	4/1/0	4/1/0	5/0/0
subtract+10	2/2/1	4/1/0	4/1/0	4/1/0
subtract+20	3/1/1	4/1/0	4/1/0	5/0/0
retrain	3/2/0	4/1/0	4/1/0	5/0/0

Operators. Table III and Fig. 2 show the outcomes. The un-unlearned MLP was flagged in 5 of 5 runs on both data sets, and so was the HAR linear model; the N-BaIoT linear model was flagged in 1 of 5, so its audit has no power (116 parameters cannot memorise 270 canaries) and any certificate there is vacuous. Cheap operators were certified about as often as retraining for the MLP on both data sets, but on N-BaIoT plain subtraction cost accuracy for the linear model (0.871 against 0.995) and replay (0.942) too, which is again why the policy has a utility guard. With 60 to 90 pairs the test is near its resolution: retraining was certified in 3 to 5 of 5 runs. The MLP is expensive on the device: retraining costs 86.5 J on HAR against 2.7 J for the linear model, and 27.4 J on N-BaIoT.

Calibration and the power check. Over 280 exactly retrained models (synthetic 40; HAR linear and MLP 60 each; N-BaIoT linear and MLP 60 each) the verifier flagged 4 (1.4%: 2 on HAR with the MLP, 2 on N-BaIoT with the linear model), below the level $\alpha = 5\%$. The un-unlearned model was flagged in 160 of 160 runs on the synthetic fleet and on HAR, but in only 35 of 60 runs for the N-BaIoT MLP at $T = 30$ and 9 of 60 for its linear model; in the remaining runs the model had not memorised the canaries enough, so the certificate that follows is true but says little. The policy therefore audits the model before unlearning and reports UNPOWERED instead of

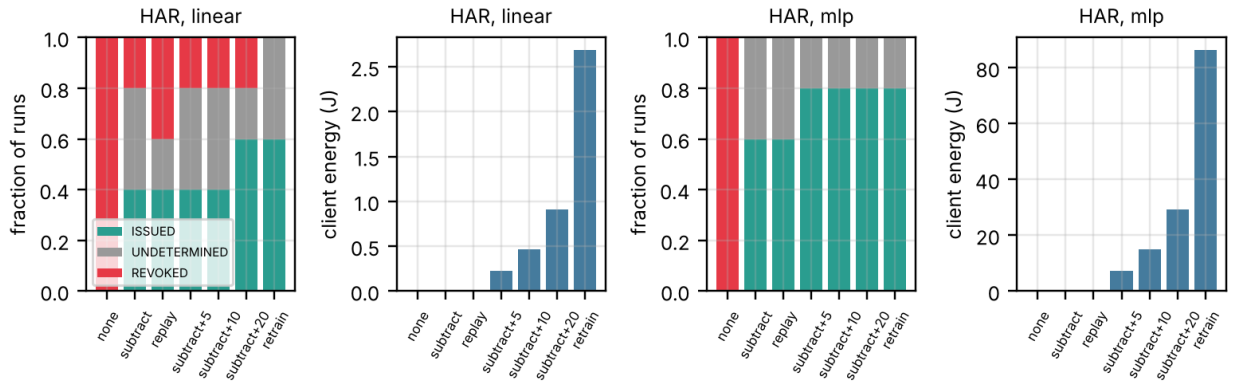


Fig. 2. UCI HAR: certificate outcomes and client energy per operator for the linear model and the MLP (5 seeds).

ISSUED when it is not flagged: on the N-BaIoT linear model it did so in 3 of 5 waves. For the MLP on N-BaIoT the ladder certified 5 of 5 waves at 0.88 J against 27.4 J for retraining (97% less; 2 rounds on average; accuracy 0.982 against 0.997), and on HAR it certified the same waves as retraining at 1.7 J against 2.7 J (linear) and 25.6 J against 86.5 J (MLP).

Poisoning. On N-BaIoT a trigger stamped on the first three features of half of the records of 2 or 5 clients implanted: the attack success rate before unlearning was 0.25 and 0.45 for the linear model and 0.36 and 0.59 for the MLP. Subtraction with 10 recovery rounds reduced it to 0.003–0.010, against 0.007–0.020 for retraining and 0.009–0.030 for a model never trained on them, at 0.2 J instead of 1.3 J (linear) and 6.6 J instead of 39.3 J (MLP, 2 poisoned clients), with accuracy unchanged. On HAR the same kind of trigger did not implant (attack success at most 7% even with triggers on up to 60 features, 8σ, 10 of 30 clients), because the sensor features separate the classes too strongly, so HAR is not informative about poisoning.

Per-client certificates. A wave of one client has only m pairs. On HAR with the MLP and $m = 60$, the un-unlearned model was flagged in 5 of 5 runs and the retrained model certified in 5 of 5 at $\epsilon = 0.2$ (subtract+20 in 4 of 5); at $m = 40$ retraining was certified in 3 of 5, and at $\epsilon = 0.3$, $m \geq 40$ sufficed (Fig. 3). On N-BaIoT the audit had no power at this model size: with $m = 60$ the un-unlearned MLP was flagged in 0 of 5 runs and with $m = 100$ in 1 of 5, and widening the hidden layer to 128 units (15 000 parameters) raised this only to 2 of 5. Canaries are 60% of a device's records (m=60), raising local compute 2.5 times, small beside the radio cost, and they use up capacity: the linear HAR model, with 562 parameters, was flagged in 3 of 5 runs at $m = 100$ against 4 of 5 at $m = 40$. Per-client certificates therefore need a model with spare capacity relative to the canary budget, and the power check shows when that is not the case.

J. Recovery policies

Table IV and Fig. 4 compare retraining, the ladder, ICR and targeted ICR (block $B = 5$, $R_{\max} = 60$, 5 seeds; targeted with $q = 0.5$ and 0.25 of the retained clients). Over all five settings ICR used 59, 39, 27, 33 and 0% less mean energy than the ladder, and 72, 60, 79, 73 and 97% less than retraining, with the same certification rate except for one more certified HAR-linear wave. Its worst-case run never cost more than retraining by more than 1% (the ladder's did in four settings, e.g. 119.6 J against 88.1 J on the HAR MLP). Targeting with $q = 0.25$ cut ICR's energy a further 62, 69, 73, 83 and 74% at nearly unchanged accuracy, but certified one wave less on the synthetic fleet and on HAR with the linear model, so q trades energy against certification. On the N-BaIoT linear model the audit is unpowered, so those certificates carry no weight.

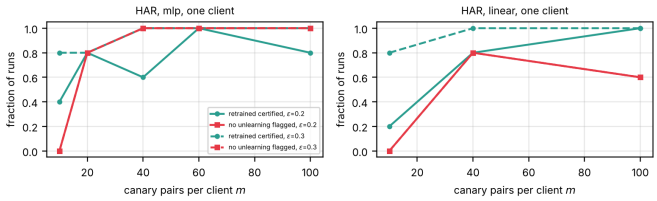


Fig. 3. One revoked client on UCI HAR: share of runs in which the retrained model is certified and the un-unlearned model is flagged, against the number of canary pairs per client (5 seeds).

TABLE IV
MEAN CLIENT ENERGY IN J / WAVES CERTIFIED (OF 5) PER POLICY. N-BAIOT LINEAR: UNPOWERED AUDIT.

	Synth. lin.	HAR lin.	HAR MLP	N-BaIoT lin.	N-BaIoT MLP
retrain	9.6/5	2.7/3	86.5/4	0.88/4	27.4/5
ladder	6.7/5	1.7/3	25.6/4	0.36/1	0.88/5
ICR	2.7/5	1.1/4	18.5/4	0.24/1	0.88/5
ICR q=0.5	1.3/4	0.40/4	9.0/4	0.11/1	0.42/5
ICR q=0.25	1.0/4	0.34/3	5.1/4	0.04/1	0.23/5

K. Real-record twins, power and scheduling

Two-tier audit. With real twins (about 100 pairs on HAR and 150 on N-BaIoT, 5 seeds) the upper bound on the advantage was 0.17–0.23 for the un-unlearned model and for every operator, so the real-record audit certified all of them; the same models were flagged by noise canaries in 5 of 5 runs (HAR, both models; N-BaIoT MLP). Over 50 further seeds the real-twin audit flagged the un-unlearned HAR and N-BaIoT MLPs in

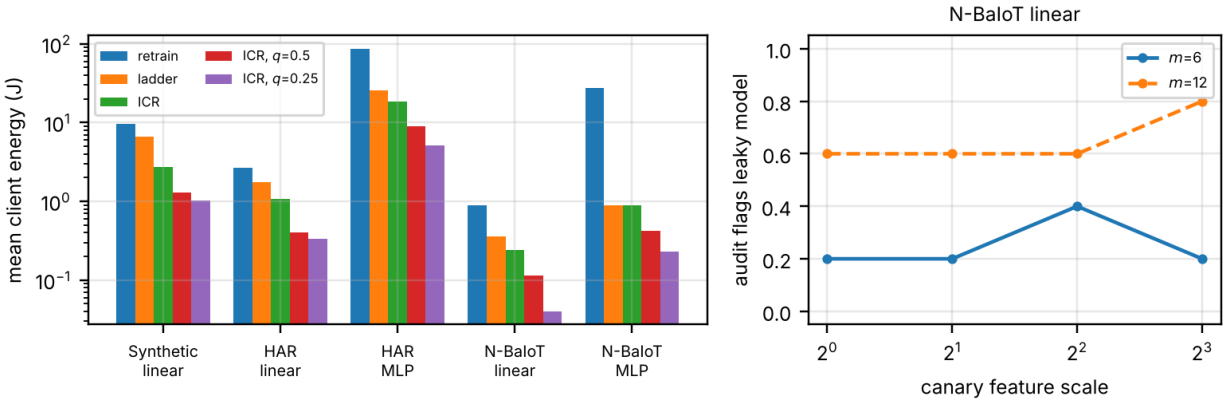


Fig. 4. Left: mean client energy per recovery policy (log scale). Right: share of runs in which the audit flags the un-unlearned N-BaIoT linear model, against canary pairs per client m and feature scale.

1 and 3 runs and the retrained ones in 1 and 2, which is the false-alarm level. These models therefore leak no real-record advantage above $\epsilon = 0.2$ even without unlearning, and the canary audit is the sensitive probe; the real-twin bound is its average-case complement, and cheap operators cannot do worse on real records than doing nothing.

Canary calibration. On N-BaIoT with the linear model the un-unlearned model was flagged in 1 or 2 of 5 runs at $m = 6$ for every scale, 3 of 5 at $m = 12$ and scales 1 to 4, and 4 of 5 at scale 8 (accuracy 0.93–0.97 at larger scales). Doubling m helped more than scale, and neither made the audit reliable for a model with 116 parameters; the HAR linear model and the N-BaIoT MLP were flagged in 5 of 5 runs at every setting.

Scheduler. A simulation over one year with Poisson revocations, $D = 30$ days, $\delta = 1$, $m = 6$, and a power target of 120 pairs used the measured wave energy of ICR on the synthetic fleet (2.7 J). At 1 request per day, waiting for 120 pairs cut energy per request from 1.8 to 0.14 J (12-fold) at a mean latency of 10.3 days and 7% underpowered waves, against 100% underpowered waves when each request is served at once; at 4 per day the saving was 5-fold at 2.8 days. At 0.2 per day every batched wave hit the deadline underpowered (mean latency 17 days), so a rare-request fleet needs more canaries per client, not longer waiting.

Baselines. We re-implemented a marker-based verifier in the style of VeriFi [7] (each leaving client stamps a random 10-feature trigger on 25% of its records and flips their labels; the check is the trigger success rate) and a loss-threshold membership test [20], and ran them on the HAR and N-BaIoT MLPs (5 seeds, wave of a third of the clients). Neither separated the un-unlearned model from the retrained one: marker success was 0.034 against 0.034 on HAR (the marker did not implant) and 0.40 against 0.43 on N-BaIoT, where the trigger flips predictions even in a model never trained on it; membership AUC was 0.44 against 0.44 and 0.43 against 0.42. The canary audit flagged the un-unlearned model in 10 of 10 runs and certified the retrained one in 9 of 10. These baselines are simple versions without tuning, so the result says that a fixed trigger or a loss threshold needs calibration that the e-process does not.

VI. Discussion and Limitations

What a certificate means. It bounds the advantage that two declared scores extract from the canary cohort of the wave on the released model, and holds at any stopping time. It is not erasure in parameter space, and it does not cover attacks outside the score class. Canaries must be planted before training, so models trained without them cannot be certified retroactively, and a provider that treats canary records specially defeats the audit. The ledger audit certifies lineage only; the statistical certificate and the lineage check answer different questions and both are reported.

Limits of the evidence. The data are a synthetic fleet and two real data sets (UCI HAR, whose classes are easy to separate, and N-BaIoT, in which we split each device into five sites ourselves), with a linear model and a one-hidden-layer MLP, 5–10 seeds, simulated clients and a modelled energy cost. We have not run deeper networks, other IoT modalities or measured device energy, and the audit's power depends on the model's capacity to memorise: it was weak for the N-BaIoT models, which is why the policy reports UNPOWERED audits. The real-record twin audit saw no leakage above ϵ even without unlearning, so only the canary audit discriminates, and the baselines are simple re-implementations. A certificate is issued per wave; certifying one client alone needed about 60 canary pairs and a model with spare capacity (HAR MLP, not N-BaIoT), and the canaries take up most of a small device buffer. The audit tests canaries, which are random records; a cheap operator can remove the influence of those and still leave subtle influence of real records, which the canary test cannot see. This is why plain subtraction was certified on HAR at no client energy, and the certificate should not be read as proof about real-sample membership. The escalation policy saves energy on average but can cost more than retraining in the worst case, so a deployment should cap the number of attempts. Sample- and class-level requests are implemented for rollback and retraining but neither tested nor evaluated here, and feature-level forgetting is not supported.

Regulatory alignment. The ledger and the certificate give a data controller a verifiable record that a revocation was

honoured; whether this satisfies a specific legal duty is a question for counsel, and we make no such claim.

VII. Conclusion

We combined a hash-chained provenance ledger, per-round state checkpoints, paired-canary e-process certificates and a cost ladder into a federated unlearning workflow whose acceptance test is statistical rather than empirical. On a synthetic fleet, smartphone sensor data and botnet traffic from nine IoT devices, with a linear model and an MLP, the audit wrongly flagged an exactly retrained model in 1.4% of runs, below its 5% level, and flagged the un-unlearned model whenever the canaries were memorised; where they were not, the power check said so. The ladder certified waves at 31 to 97% lower mean energy than retraining, and batching revocations cut per-client cost by an order of magnitude. A single client could be certified with 60 canary pairs on a HAR MLP. Next steps are deeper networks, other IoT modalities, measured device energy, a canary design that tests real records, and attacks that adapt to the audit.

References

[1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. AISTATS*, 2017.

[2] European Parliament and Council, "Regulation (EU) 2016/679 (general data protection regulation), art. 17," 2016.

[3] G. Liu, X. Ma, Y. Yang, C. Wang, and J. Liu, "FedEraser: Enabling efficient client-level data removal from federated learning models," in *Proc. IEEE/ACM Int. Symp. Quality of Service (IWQoS)*, 2021.

[4] A. Halimi, S. Kadhe, A. Rawat, and N. Baracaldo, "Federated unlearning: How to efficiently erase a client in FL?" *arXiv preprint arXiv:2207.05521*, 2022.

[5] C. Wu, S. Zhu, and P. Mitra, "Federated unlearning with knowledge distillation," *arXiv preprint arXiv:2201.09441*, 2022.

[6] Z. Liu, Y. Jiang, J. Shen, M. Peng, K.-Y. Lam, X. Yuan, and X. Liu, "A survey on federated unlearning: Challenges, methods, and future directions," *ACM Computing Surveys*, 2024.

[7] X. Gao, X. Ma, J. Wang, Y. Sun, B. Li, S. Ji, P. Cheng, and J. Chen, "VeriFi: Towards verifiable federated unlearning," *IEEE Trans. Dependable and Secure Computing*, 2024.

[8] P. Goswami, M. K. Islam, and A. Yeafi, "PrivEraserVerify: Efficient, private, and verifiable federated unlearning," *arXiv preprint arXiv:2604.12348*, 2026.

[9] X. Zuo, M. Wang, T. Zhu, L. Zhang, S. Yu, and W. Zhou, "Federated learning with blockchain-enhanced machine unlearning: A trustworthy approach," *arXiv preprint arXiv:2405.20776*, 2024.

[10] "BlockFUL: Enabling unlearning in blockchained federated learning," *arXiv preprint arXiv:2402.16294*, 2024.

[11] Q.-V. Dang, "FORTRESS-FL: Federated operator resilient trustworthy resource efficient secure slice learning," Software and manuscript, https://github.com/vinhqdang/fortress-fl, 2025.

[12] ——, "VOUCH: Anytime-valid certificates for machine unlearning via paired-canary hypothesis-betting," *Machine Learning*, 2026, accepted.

[13] D. Anguita, A. Ghio, L. Oneto, X. Parra, and J. L. Reyes-Ortiz, "A public domain dataset for human activity recognition using smartphones," in *Proc. European Symp. Artificial Neural Networks (ESANN)*, 2013.

[14] Y. Meidan, M. Bohadana, Y. Mathov, Y. Mirsky, A. Shabtai, D. Breitenbacher, and Y. Elovici, "N-BaIoT—network-based detection of IoT botnet attacks using deep autoencoders," *IEEE Pervasive Computing*, vol. 17, no. 3, 2018.

[15] "Privacy-preserving federated unlearning with certified client removal," *arXiv preprint arXiv:2404.09724*, 2024.

[16] "FedQUIT: On-device federated unlearning via a quasi-competent virtual teacher," *arXiv preprint arXiv:2408.07587*, 2024.

[17] X. Cao, J. Jia, Z. Zhang, and N. Z. Gong, "FedRecover: Recovering from poisoning attacks in federated learning using historical information," in *Proc. IEEE Symp. Security and Privacy*, 2023.

[18] L. Bourtoule, V. Chandrasekaran, C. A. Choquette-Choo, H. Jia, A. Travers, B. Zhang, D. Lie, and N. Papernot, "Machine unlearning," in *Proc. IEEE Symp. Security and Privacy*, 2021.

[19] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, "Membership inference attacks against machine learning models," in *Proc. IEEE Symp. Security and Privacy*, 2017.

[20] S. Yeom, I. Giacomelli, M. Fredrikson, and S. Jha, "Privacy risk in machine learning: Analyzing the connection to overfitting," in *Proc. IEEE Computer Security Foundations Symp.*, 2018.

[21] A. Thudi, H. Jia, I. Shumailov, and N. Papernot, "On the necessity of auditable algorithmic definitions for machine unlearning," in *Proc. USENIX Security Symp.*, 2022.

[22] C. Guo, T. Goldstein, A. Hannun, and L. van der Maaten, "Certified data removal from machine learning models," in *Proc. ICML*, 2020.

[23] R. C. Merkle, "A digital signature based on a conventional encryption function," in *Proc. CRYPTO*, 1987.

[24] A. Ramdas, P. Grünwald, V. Vovk, and G. Shafer, "Game-theoretic statistics and safe anytime-valid inference," *Statistical Science*, vol. 38, no. 4, 2023.

[25] A. Ramdas and R. Wang, *Hypothesis Testing with E-values*. Foundations and Trends in Statistics (to appear), 2024.

[26] T. P. Pedersen, "Non-interactive and information-theoretic secure verifiable secret sharing," in *Proc. CRYPTO*, 1991.

[27] P. Blanchard, E. M. E. Mhamdi, R. Guerraoui, and J. Stainer, "Machine learning with adversaries: Byzantine tolerant gradient descent," in *Proc. NeurIPS*, 2017.

[28] C. Dwork and A. Roth, *The Algorithmic Foundations of Differential Privacy*. Now Publishers, 2014.